Smart Health Monitoring Band using IoT and MQTT

Title

Smart Health Monitoring System for Multiple Patients using IoT, MQTT, Raspberry Pi and Web Dashboard

Abstract

The Smart Health Monitoring System is an Internet of Things (IoT) based solution designed to continuously monitor the health parameters of multiple patients remotely. In this project, wearable health bands are simulated using the online platform Wokwi Simulator. The simulator generates health data such as heart rate and body temperature for ten patients.

The generated data is transmitted using the lightweight communication protocol MQTT through a public broker. A monitoring system running on a Raspberry Pi subscribes to the incoming sensor data and processes it in real time. The system analyzes the received values and detects abnormal conditions such as high heart rate.

All health data is stored in a local text file for record keeping and future analysis. A web-based dashboard developed using Flask allows users to monitor patients through a dropdown menu. The system also generates alerts whenever a patient's health parameters exceed predefined threshold values.

This project demonstrates how IoT technology can be used to enable remote health monitoring and early detection of medical risks.

Problem Statement

Many patients, especially elderly individuals and people with chronic diseases, require continuous monitoring of vital health parameters such as heart rate and body temperature. Traditional healthcare systems require frequent hospital visits, which can be inconvenient and time consuming.

In situations where patients are located far from hospitals or medical staff, it becomes difficult to continuously monitor their health conditions. Lack of real-time monitoring can lead to delayed detection of medical emergencies.

Therefore, there is a need for an IoT-based system that can remotely monitor patient health parameters in real time, analyze abnormal conditions, and notify healthcare providers immediately.

Objectives

The main objectives of this project are:

- To simulate wearable health monitoring devices for multiple patients.
- To generate health sensor data using Wokwi Simulator.
- To transmit sensor data using the MQTT communication protocol.
- To receive and analyze health data using a Raspberry Pi monitoring system.
- To store health data records in a file for analysis and tracking.
- To design a web-based dashboard using Flask.
- To generate alerts whenever a patient's heart rate exceeds the safe threshold.

Architecture Diagram

System Architecture

```
Health Sensor Simulation
(Wokwi Simulator)
     |
     | MQTT Publish
     ▼
MQTT Broker
(broker.hivemq.com)
     |
     | MQTT Subscribe
     ▼
Raspberry Pi Monitoring System
     |
     ├── Data Analysis
     ├── Threshold Detection
     ├── Data Storage (Text File)
```

|
▼
Flask Web Server
|
▼
Web Dashboard
(Dropdown for 10 Patients)
|
▼
Alert Notification

Architecture Explanation

1. **Sensor Simulation Layer**
   The health sensors are simulated using Wokwi Simulator which generates heart rate and temperature values for ten patients.
2. **Communication Layer**
   Data is transmitted using the MQTT protocol via the public broker **broker.hivemq.com**.
3. **Processing Layer**
   A monitoring program running on a Raspberry Pi subscribes to the MQTT topic and processes incoming data.
4. **Application Layer**
   A web application built using Flask displays patient data and alerts.

Technologies Used

Technology	Purpose
Python	Data processing and monitoring
MQTT Protocol	Lightweight IoT communication
HiveMQ Public Broker	MQTT message broker
Wokwi Simulator	Sensor simulation
Raspberry Pi	Monitoring and processing
Flask	Web server and dashboard

Technology	Purpose
HTML	Frontend webpage
GitHub	Source code version control

Implementation (Git)

The implementation of the project consists of the following modules.

1. Sensor Simulation

Ten patients are simulated using Wokwi Simulator. The simulator generates random health values including:

- Heart Rate (60–140 bpm)
- Temperature (36–38.5 °C)

The simulated ESP32 device publishes these values to the MQTT topic:

healthband/data

MQTT Communication

The simulated devices publish health data to the public MQTT broker **broker.hivemq.com**. The monitoring system subscribes to this topic to receive patient data.

Raspberry Pi Monitoring System

A Python program running on the Raspberry Pi performs the following tasks:

• Receives MQTT messages
• Extracts patient data
• Detects abnormal heart rate
• Stores the data in a text file

Example stored data:

2026-03-14 10:22:11, Aarav Sharma, 92, 36.8
2026-03-14 10:22:13, Rohan Verma, 128, 37.1

Alert Detection

The system checks whether the received values exceed predefined limits.

Parameter	Threshold	Alert
Heart Rate	>120 bpm	High Heart Rate Alert
Temperature	>38 °C	Fever Alert

When a threshold is crossed, the system prints an alert message and notifies the web dashboard.

Web Dashboard

A web server created using Flask provides a monitoring interface.

Dashboard features:

- Dropdown list of 10 patients
- Display of heart rate and temperature
- Alert notifications for abnormal values

The dashboard can be accessed through VNC using:

http://localhost:5000

Results / Output

The system successfully simulated ten wearable health monitoring devices and transmitted sensor data through MQTT.

Sample Output (Raspberry Pi Terminal)

Aarav Sharma 85 36.7
Diya Patel 92 36.8
Rohan Verma 128 37.2
ALERT: High Heart Rate detected for Rohan Verma

Sample Data File

2026-03-14 11:10:22, Aarav Sharma, 82, 36.7
2026-03-14 11:10:24, Diya Patel, 90, 36.8

Dashboard Output

The web interface shows:

- Patient selection dropdown
- Real-time health values
- Alert messages for abnormal conditions

Challenges Faced

During the development of the project, several challenges were encountered:

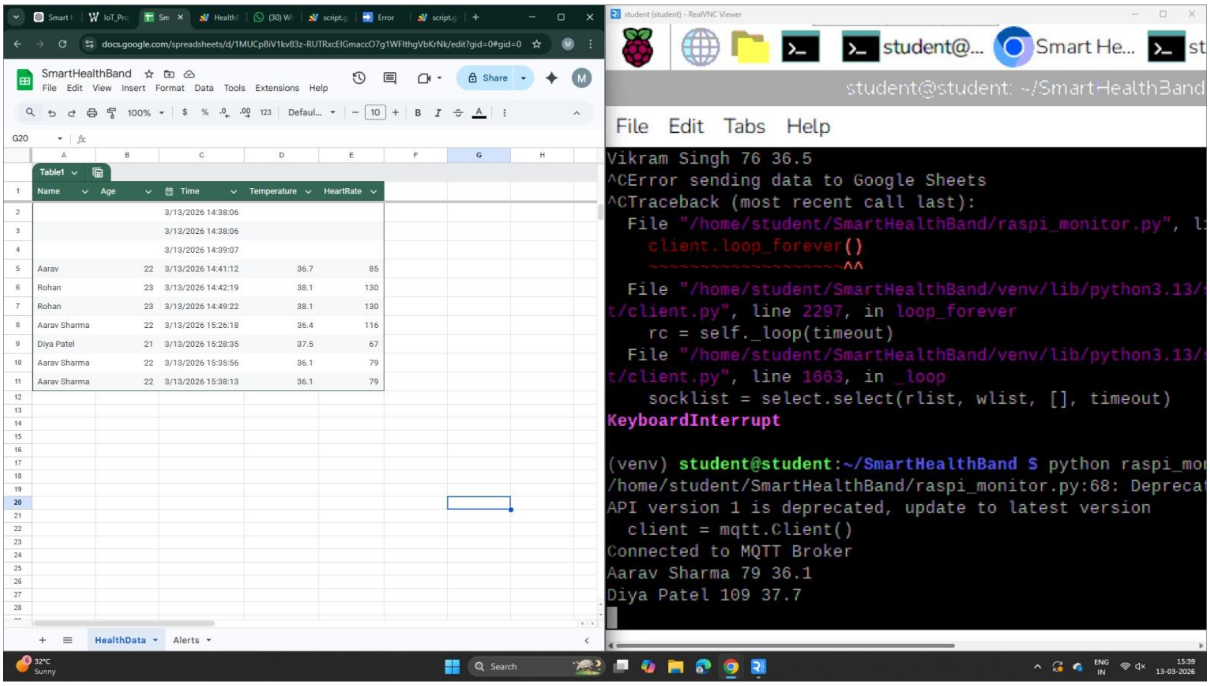
- Setting up MQTT communication between the simulator and the monitoring system.
- Handling continuous real-time data streams.
- Simulating realistic health data values.
- Designing a simple and responsive web dashboard.
- Managing multiple patient data streams simultaneously.

Future Improvements

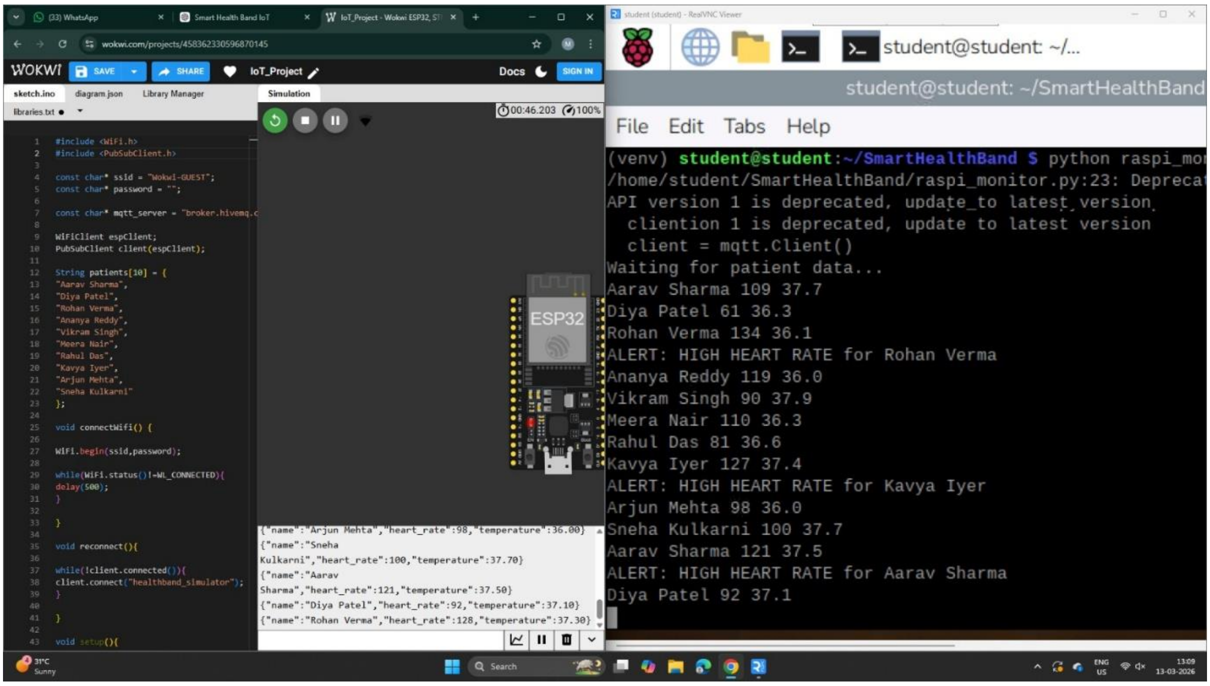
The system can be enhanced with several additional features:

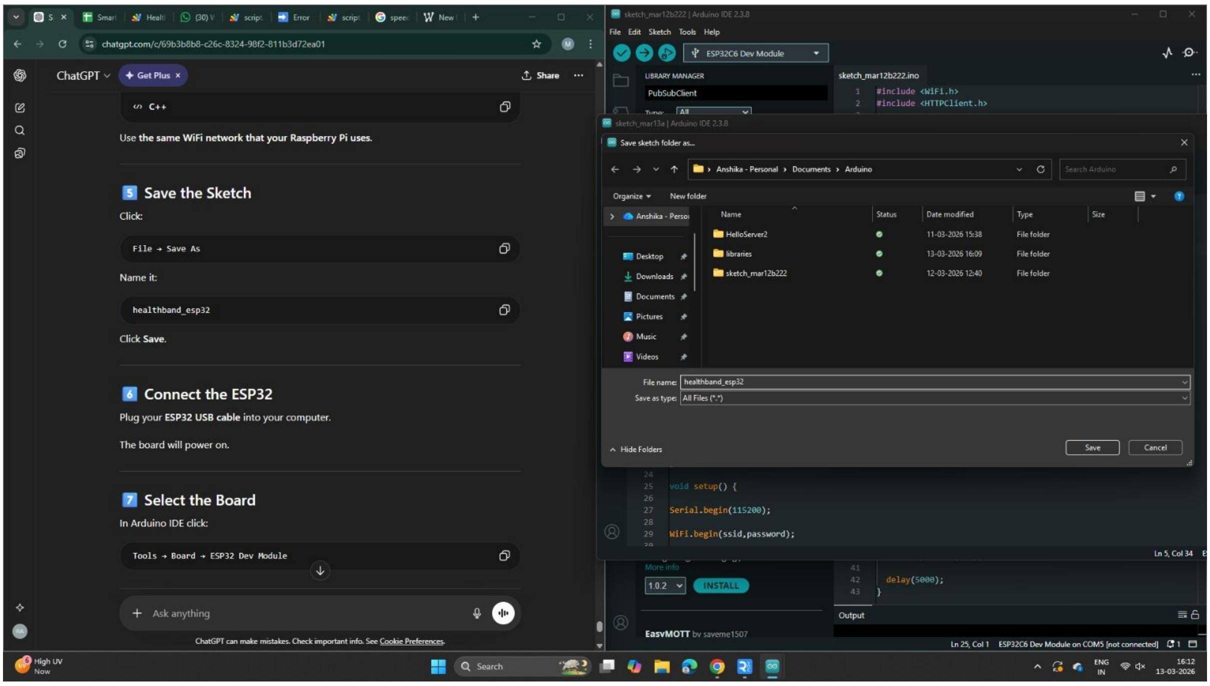
- Integration with real wearable sensors instead of simulated devices.
- Development of a mobile application for remote monitoring.
- Implementation of cloud storage for long-term patient data.
- Integration with hospital systems for automatic medical alerts.

• Addition



of machine learning algorithms to predict health risks.




chatgpt.com/c/69b3b8b8-c26c-8324-98f2-811b3d72ea01
ChatGPT
Get Plus
Share
C++
Use the same WiFi network that your Raspberry Pi uses.
5 Save the Sketch
Click:
File → Save As
Name it:
healthband_esp32
Click Save.
6 Connect the ESP32
Plug your ESP32 USB cable into your computer.
The board will power on.
7 Select the Board
In Arduino IDE click:
Tools → Board → ESP32 Dev Module
Ask anything
ChatGPT can make mistakes. Check important info. See Cookie Preferences.
sketch_mar12b222 | Arduino IDE 2.3.8
File Edit Sketch Tools Help
ESP32C6 Dev Module
LIBRARY MANAGER
PubSubClient
sketch_mar12b222.ino
#include <WiFi.h>
#include <HTTPClient.h>
sketch_mar13a | Arduino IDE 2.3.8
Save sketch folder as...
Anshika - Personal › Documents › Arduino
Search Arduino
Organize
New folder
Name
Status
Date modified
Type
Size
HelloServer2
11-03-2026 15:38
File folder
libraries
13-03-2026 16:09
File folder
sketch_mar12b222
12-03-2026 12:40
File folder
Desktop
Downloads
Documents
Pictures
Music
Videos
File name: healthband_esp32
Save as type: All Files (*.*)
Hide Folders
Save
Cancel
void setup() {
Serial.begin(115200);
WiFi.begin(ssid,password);
delay(5000);
}
Ln 5, Col 34
More info
1.0.2
INSTALL
Output
EasyMQTT by saveme1507
Ln 25, Col 1 ESP32C6 Dev Module on COM5 [not connected]
High UV
Now
Search
ENG
IN
18:12
13-03-2026